MANUAL DIDÁTICO • PARA INICIANTES

Conectando o Gemini ao Claude

Aprenda, passo a passo, a dar ao Claude uma busca na internet com fontes citadas.

14 aulas curtas • pontos de conferência • exercício com gabarito

Autoria e criação do projeto: Hudson Travassos (@pro.hudsontravassos)

Repositório: github.com/wall-e-assessor/gemini-grounding-mcp

Licença: MIT • Outubro de 2026

Sumário

- Aula 0. Combinado entre nós
- Aula 1. Entendendo a ideia antes de mexer em qualquer coisa
- Aula 2. Pequeno dicionário
- Aula 3. O que você precisa ter em mãos
- Aula 4. Aprendendo a usar o Terminal (sem medo)
- Aula 5. Instalando o Node.js
- Aula 6. Pegando a sua chave do Gemini
- Aula 7. Baixando o projeto
- Aula 8. Montando o projeto (install e build)
- Aula 9. Descobrindo o caminho da pasta
- Aula 10. A conexão: apresentando o Gemini ao Claude
- Aula 11. Testando
- Aula 12. Quando algo dá errado
- Aula 13. Manutenção: trocar chave, atualizar, remover
- Aula 14. Segurança: as regras de ouro
- Exercício de fixação
- Resumo de bolso
- Créditos

Aula 0. Combinado entre nós

Antes de começar, três combinados de professor para aluno:

1. **Leia o passo inteiro antes de executar.** Cada passo tem um "porquê". Entender o porquê evita 90% dos erros.
2. **Siga a ordem.** Cada aula depende da anterior. Pular etapa é como tentar ligar a TV antes de ligar a tomada.
3. **Use os pontos de conferência.** Ao final de cada aula existe um quadro ✔ **Confira**. Só avance quando ele estiver certo.

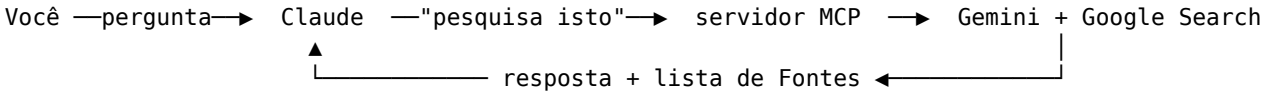
Tempo estimado: **de 15 a 30 minutos** na primeira vez. Custo: **zero**, usando o nível gratuito do Google AI Studio (veja a observação sobre limites na Aula 6).

Aula 1. Entendendo a ideia antes de mexer em qualquer coisa

A história em uma imagem

Imagine um escritório:

- O **Claude** é um assistente brilhante, que escreve, analisa e raciocina muito bem. Ele trabalha numa sala sem janela: nem sempre sabe o que aconteceu ontem no mundo.
- O **Gemini**, do Google, tem um **telefone direto com o Google Search**. Ele consegue pesquisar na internet agora e dizer **de onde** tirou cada informação.
- O **servidor MCP** que vamos instalar é um **ramal telefônico** entre a sala do Claude e a sala do Gemini. Quando o Claude precisar de algo atual, ele "liga" para o Gemini, que pesquisa e responde com as fontes.



Por que este ramal é especial

O autor escreveu o servidor com dois cuidados:

- **Resposta enxuta.** O Google devolve, junto com a resposta, um bloco de código visual (HTML/CSS) da caixa de pesquisa. Esse bloco não tem utilidade para o Claude e gasta espaço. O servidor descarta o bloco e entrega só **o texto e a lista de fontes**.
- **Linha limpa.** O servidor conversa com o Claude por um canal chamado `stdout`. Qualquer mensagem extra nesse canal "embaralharia a ligação". Por isso todo aviso interno vai para outro canal (`stderr`).

Você não precisa decorar isso. Basta saber: **o projeto foi feito para ser econômico e estável.**

Aula 2. Pequeno dicionário

Palavra	O que significa, em português simples
Claude Code	O Claude usado pelo Terminal do computador. É nele que vamos plugar o Gemini.
MCP (Model Context Protocol)	Um "padrão de tomada". Qualquer ferramenta que siga esse padrão pode ser plugada no Claude.
Servidor MCP	O "aparelho" que plugamos nessa tomada. Aqui, o `gemini-grounding-mcp`.
Grounding	"Ancorar" a resposta em pesquisas reais, com fontes que você pode conferir.
Chave de API (API key)	Uma senha pessoal que o Google te dá para usar o Gemini. Funciona como um **crachá**.
Terminal	Uma janela onde você digita comandos em vez de clicar. No Windows também se chama **Prompt de Comando** ou **PowerShell**.
Node.js	O "motor" que faz o servidor funcionar. Sem ele, o projeto é só um monte de arquivos parados.
npm	O "entregador" que vem junto com o Node.js e baixa as peças de que o projeto precisa.
Build	"Montar" o projeto: transformar o código-fonte em um programa pronto para rodar.
Caminho (path)	O endereço de uma pasta ou arquivo no computador, como `C:\Projetos\gemini-grounding-mcp`.

Aula 3. O que você precisa ter em mãos

Faça um checklist antes de começar:

- ☐ Um computador **Windows** ou **Mac** (Linux também funciona, com os mesmos comandos do Mac).
- ☐ O **Claude Code** já instalado e funcionando. Para testar, abra o Terminal e digite `claude --version`. Se aparecer um número, está ok.
- ☐ Uma **conta Google** (Gmail serve).
- ☐ Conexão com a internet.
- ☐ Um bloco de notas aberto para anotar **duas coisas**: o **caminho da pasta** do projeto e a **sua chave** (esta, só temporariamente; veja a Aula 14).

> **Dica de professor:** se o Claude Code ainda não estiver instalado, resolva isso primeiro seguindo a documentação oficial: https://code.claude.com/docs

Aula 4. Aprendendo a usar o Terminal (sem medo)

O Terminal assusta à primeira vista, mas a regra é simples: **você digita uma linha, aperta Enter, e o computador responde.**

Como abrir

- **Windows:** aperte a tecla **Windows**, digite `cmd` ou `PowerShell` e clique no programa.
- **Mac:** aperte **Command + Espaço**, digite `Terminal` e aperte Enter.

Quatro regras de ouro do Terminal

1. **Copie e cole os comandos** deste manual em vez de digitar. Isso evita erro de digitação.
   - Windows: colar com **Ctrl + V** ou com o **botão direito do mouse**.
   - Mac: colar com **Command + V**.
2. **Espere terminar.** Quando o comando acaba, o cursor volta a piscar numa linha nova, pronto para outro comando.
3. **Cada comando é uma linha só.** Em telas estreitas ou no PDF, um comando longo pode aparecer quebrado em duas linhas. Ao digitar ou colar, junte tudo numa linha única e só então aperte Enter.
4. **Mensagens em inglês são normais.** Só se preocupe se aparecer a palavra **ERROR** (erro) ou **not found / não é reconhecido**. Avisos com **WARN** (aviso) em geral podem ser ignorados.

✔ Confira

Abra o Terminal, digite `claude --version` e aperte Enter. Apareceu um número de versão? Ótimo, você já sabe usar o Terminal.

Aula 5. Instalando o Node.js

Por que: o servidor foi escrito em TypeScript/JavaScript. O Node.js é quem executa esse tipo de programa. O projeto exige **Node.js 18 ou superior**.

Passo a passo

1. Acesse **https://nodejs.org**.
2. Baixe a versão marcada como **LTS** ("Long Term Support", a versão estável recomendada).
3. Abra o arquivo baixado e clique em **Avançar / Next** até o fim, mantendo as opções padrão.
4. **Feche o Terminal e abra de novo.** O Terminal só "enxerga" programas instalados depois que é reaberto.

✔ Confira

No Terminal novo, digite:

```
node --version
```

Deve aparecer algo como `v22.x.x` ou `v24.x.x`. Qualquer número a partir de `v18` serve.

Teste também o entregador:

```
npm --version
```

Apareceu um número? Siga em frente.

Aula 6. Pegando a sua chave do Gemini

Por que: o Google precisa saber **quem** está usando o Gemini. A chave é o seu crachá de identificação.

Passo a passo

1. Acesse **https://aistudio.google.com/apikey**.
2. Entre com a sua conta Google.
3. Clique em **"Create API key"** (Criar chave de API). Se o site pedir para escolher ou criar um projeto, aceite a opção sugerida.
4. Copie a chave gerada. Ela é uma sequência longa de letras e números.
5. Cole a chave **temporariamente** no seu bloco de notas. Você vai usá-la na Aula 10.

⚠ **Atenção, aluno:** a chave é **pessoal e secreta**, como a senha do banco. Nunca envie por WhatsApp, nunca poste em grupo, nunca mostre em print de tela. Quem tiver a sua chave usa a sua cota em seu nome.

Sobre gratuidade e limites: o Google AI Studio oferece um nível gratuito, com limites de uso que o Google pode alterar. Confira os limites vigentes na página oficial de preços do Gemini antes de usar em volume.

✔ Confira

Você tem no bloco de notas uma sequência longa de caracteres copiada do AI Studio? Então está pronto.

Aula 7. Baixando o projeto

Por que: precisamos trazer para o seu computador o código que o autor publicou.

Existem dois jeitos. **Para iniciantes, recomendo a Opção A.**

Opção A: baixar o ZIP (mais simples)

1. Acesse **https://github.com/wall-e-assessor/gemini-grounding-mcp**.
2. Clique no botão verde **"Code"**.
3. Clique em **"Download ZIP"**.
4. Encontre o arquivo baixado (geralmente na pasta **Downloads**) e **extraia** (descompacte):
   - Windows: botão direito no arquivo, **"Extrair tudo..."**.
   - Mac: dois cliques no arquivo.
5. **Mova a pasta extraída para um lugar fácil e definitivo**, por exemplo:
   - Windows: `C:\Projetos\gemini-grounding-mcp`
   - Mac: `/Users/SEU_USUARIO/Projetos/gemini-grounding-mcp`

> **Por que mover?** O Claude vai procurar o servidor sempre no mesmo endereço. Se você deixar em Downloads e um dia limpar essa pasta, a conexão quebra. Escolha o lugar agora e não mexa mais.

> **Detalhe do ZIP:** às vezes a pasta extraída vem com o nome `gemini-grounding-mcp-main`. Pode renomear para `gemini-grounding-mcp` ou manter, desde que você use o nome certo nos próximos passos.

Opção B: usando o git (para quem já tem o git instalado)

No Terminal, vá até a pasta onde quer guardar o projeto e rode:

```
git clone https://github.com/wall-e-assessor/gemini-grounding-mcp.git
```

✔ Confira

Abra a pasta do projeto. Dentro dela você deve ver, entre outros, os arquivos `package.json`, `README.md` e uma pasta chamada `src`.

Aula 8. Montando o projeto (install e build)

Por que: o que você baixou é a "receita". Agora vamos buscar os ingredientes (`npm install`) e cozinhar o prato (`npm run build`).

8.1. Abrir o Terminal dentro da pasta do projeto

Esse é o ponto em que mais gente erra. O comando precisa rodar **dentro** da pasta certa.

- **Windows:** abra a pasta do projeto no Explorador de Arquivos, clique na **barra de endereço** lá em cima, apague o que estiver escrito, digite `cmd` e aperte Enter. Um Terminal abre já dentro da pasta.
- **Mac:** abra o Terminal, digite `cd ` (com um espaço depois), **arraste a pasta do projeto** do Finder para dentro da janela do Terminal e aperte Enter.

Para ter certeza de que está no lugar certo, digite:

- Windows: `dir`
- Mac: `ls`

Você deve ver o arquivo `package.json` na lista. Se não vir, você está na pasta errada.

8.2. Buscar os ingredientes

```
npm install
```

Pode demorar de alguns segundos a alguns minutos. Mensagens com **WARN** são normais.

8.3. Montar o programa

```
npm run build
```

Esse comando cria uma pasta nova chamada `dist`, e dentro dela o arquivo `index.js`. Esse arquivo é o servidor pronto.

✔ Confira

Abra a pasta do projeto: existe agora uma pasta `dist` com o arquivo `index.js` dentro? Então o prato está pronto.

Aula 9. Descobrindo o caminho da pasta

Por que: na próxima aula você vai dizer ao Claude **onde** está o servidor. Para isso, precisa do endereço completo (o "caminho absoluto") do arquivo `dist/index.js`.

Windows

1. Abra a pasta `dist` no Explorador de Arquivos.
2. Segure **Shift**, clique com o botão direito no arquivo `index.js` e escolha **"Copiar como caminho"**.
3. Cole no bloco de notas. Vai ficar parecido com:

```
"C:\Projetos\gemini-grounding-mcp\dist\index.js"
```

Mac

1. Abra a pasta `dist` no Finder.
2. Clique com o botão direito no arquivo `index.js`, segure a tecla **Option** e escolha **"Copiar 'index.js' como Nome de Caminho"**.
3. Cole no bloco de notas. Vai ficar parecido com:

```
/Users/SEU_USUARIO/Projetos/gemini-grounding-mcp/dist/index.js
```

> **Dica de professor:** se o caminho tiver **espaços** (por exemplo, `Meus Projetos`), mantenha as **aspas** em volta dele no comando da próxima aula.

✔ Confira

No seu bloco de notas existem agora duas anotações: **a chave** (Aula 6) e **o caminho do** `index.js` (esta aula).

Aula 10. A conexão: apresentando o Gemini ao Claude

Este é o momento principal. Vamos registrar o servidor no Claude Code com **um único comando**.

Entendendo o comando, peça por peça

Antes de executar, veja o que cada parte faz:

```
claude mcp add --scope user --env GEMINI_API_KEY=SUA_CHAVE_AQUI --transport stdio
    gemini-grounding -- node "CAMINHO_DO_INDEX_JS"
```

Parte do comando	O que ela diz ao Claude
`claude mcp add`	"Quero adicionar uma ferramenta nova."
`--scope user`	"Deixe essa ferramenta disponível em **todos** os meus projetos."
`--env GEMINI_API_KEY=SUA_CHAVE_AQUI`	"Entregue este crachá ao servidor quando ele ligar."
`--transport stdio`	"A conversa com o servidor será local, pelo canal padrão do computador."
`gemini-grounding`	O **apelido** da ferramenta. É por esse nome que você vai encontrá-la depois.
`--`	Uma "linha divisória": tudo que vem depois é o comando que liga o servidor.
`node "CAMINHO_DO_INDEX_JS"`	"Para ligar o servidor, use o Node.js para rodar este arquivo."

Por que a ordem importa? As opções (`--scope`, `--env`, `--transport`) vêm **antes** do apelido. A documentação oficial do Claude Code avisa que, se o apelido vier logo depois de `--env`, o Claude confunde o apelido com mais uma variável e recusa o comando. Por isso o `--transport stdio` fica entre o `--env` e o apelido.

Agora, monte o seu comando

Pegue o modelo acima e faça **duas trocas** no bloco de notas:

1. Troque `SUA_CHAVE_AQUI` pela sua chave (Aula 6). **Sem espaços** e **sem aspas** em volta da chave.
2. Troque `CAMINHO_DO_INDEX_JS` pelo caminho que você copiou (Aula 9).

Exemplo no Mac (com dados fictícios):

```
claude mcp add --scope user --env GEMINI_API_KEY=SUA_CHAVE_AQUI --transport stdio
    gemini-grounding -- node
    "/Users/SEU_USUARIO/Projetos/gemini-grounding-mcp/dist/index.js"
```

Exemplo no Windows (com dados fictícios):

```
claude mcp add --scope user --env GEMINI_API_KEY=SUA_CHAVE_AQUI --transport stdio
    gemini-grounding -- node "C:\Projetos\gemini-grounding-mcp\dist\index.js"
```

Se o Windows não encontrar o `node`: troque a palavra `node` pelo caminho completo do programa, entre aspas: `"C:\Program Files\nodejs\node.exe"`.

Copie o **seu** comando montado, cole no Terminal e aperte Enter.

> **Logo depois:** apague a chave do bloco de notas. Ela já ficou guardada na configuração do Claude.

Alternativa: configurar por arquivo JSON

Alguns clientes MCP (e usuários mais experientes) preferem escrever a configuração em formato JSON. O conteúdo equivalente é:

```
{
  "mcpServers": {
    "gemini-grounding": {
      "command": "node",
      "args": ["/caminho/absoluto/para/gemini-grounding-mcp/dist/index.js"],
      "env": {
        "GEMINI_API_KEY": "SUA_CHAVE_AQUI"
      }
    }
  }
}
```

Para quem está começando, o comando `claude mcp add` é mais seguro: ele grava tudo no lugar certo.

✔ Confira

No Terminal, rode:

```
claude mcp list
```

Procure a linha `gemini-grounding`. O esperado é ver `✔ Connected` (conectado).

Aula 11. Testando

1. **Feche** o Claude Code e **abra de novo**, para ele carregar a ferramenta nova.
2. Dentro do Claude Code, digite `/mcp` e aperte Enter. O `gemini-grounding` deve aparecer na lista.
3. Agora peça, com suas palavras, algo como:

> Faça uma busca na web com o Gemini sobre a previsão do tempo para amanhã em São Luís.

4. O Claude vai pedir permissão para usar a ferramenta `search_with_grounding`. Autorize.
5. A resposta deve terminar com uma seção `Fontes:`, numerada, com o título e o endereço de cada site consultado.

Exemplo do formato da resposta

```
(texto da resposta do Gemini...)

Fontes:
1. Nome do site A — https://site-a.exemplo
2. Nome do site B — https://site-b.exemplo
```

Dica de uso: o campo "contexto"

A ferramenta aceita dois campos:

Campo	Obrigatório	Para que serve
`query`	sim	A pergunta em si.
`context`	não	Uma orientação extra, por exemplo: "responda em português e priorize fontes oficiais do governo".

Você não precisa preencher esses campos à mão: basta pedir ao Claude em linguagem natural e ele monta a chamada.

✔ Confira

Recebeu uma resposta com a lista de **Fontes**? **Parabéns: a conexão está funcionando.**

Aula 12. Quando algo dá errado

Aqui estão os problemas mais comuns, com o **sintoma**, a **causa** e o **remédio**.

Sintoma	Causa provável	Remédio
`'node' não é reconhecido` ou `command not found: node`	O Node.js não foi instalado, ou o Terminal foi aberto antes da instalação.	Refaça a Aula 5 e **abra um Terminal novo**.
`npm install` mostra **ERROR** de permissão	Pasta protegida pelo sistema.	Mova o projeto para uma pasta sua (ex.: `C:\Projetos` ou `~/Projetos`) e repita a Aula 8.
`claude mcp list` mostra `✘ Failed to connect`	Caminho do `index.js` errado, ou a Aula 8 não terminou.	Confirme que `dist/index.js` existe. Rode `claude mcp get gemini-grounding` e leia a linha `Issue:` (essa tela mostra a sua chave: não tire print). Se o caminho estiver errado, remova (Aula 13) e refaça a Aula 10.
Mensagem `ERRO: a variável de ambiente GEMINI_API_KEY não está definida`	O comando da Aula 10 foi executado sem a parte `--env`, ou com a chave vazia.	Remova o servidor (Aula 13) e refaça a Aula 10 com atenção à chave.
O Claude recusa o comando dizendo que o nome é inválido	O apelido ficou logo depois do `--env`.	Use exatamente a ordem ensinada na Aula 10, com `--transport stdio` antes do apelido.
A busca responde com erro de **chave inválida** ou **permissão negada**	Chave copiada pela metade, com espaço, ou revogada.	Gere uma chave nova (Aula 6), remova o servidor e refaça a Aula 10.
A busca responde com erro de **cota** ou **limite** (*quota*, *rate limit*, código 429)	Você passou do limite do nível gratuito naquele período.	Aguarde e tente mais tarde, ou consulte os planos na página oficial de preços do Gemini.
A busca responde que o **modelo não existe** ou foi **descontinuado**	O Google aposenta modelos de tempos em tempos. O padrão do projeto é `gemini-2.5-flash`.	Escolha um modelo atual na documentação oficial do Gemini e informe-o com uma segunda variável: acrescente `--env GEMINI_MODEL=NOME_DO_MODELO` ao comando da Aula 10.
A ferramenta não aparece no Claude	O Claude Code não foi reiniciado.	Feche e abra o Claude Code. Rode `/mcp` para conferir.

> **Dica de professor:** quando pedir ajuda a alguém, mande **a mensagem de erro** e **o comando que você usou**, mas **troque a chave por** `SUA_CHAVE_AQUI` antes de enviar.

Aula 13. Manutenção: trocar chave, atualizar, remover

Ver os detalhes da conexão

```
claude mcp get gemini-grounding
```

> ⚠ Esse comando exibe a sua chave **por extenso**, na linha `Environment`. Não tire print dessa tela e não a compartilhe.

Remover a conexão

```
claude mcp remove gemini-grounding --scope user
```

Trocar a chave

Não existe um comando de "editar". O caminho mais simples é **remover e adicionar de novo**:

1. `claude mcp remove gemini-grounding --scope user`
2. Refaça a Aula 10 com a chave nova.

Atualizar o projeto quando o autor publicar melhorias

- **Se usou o ZIP (Opção A):** baixe o ZIP novo, substitua os arquivos **na mesma pasta** e rode de novo `npm install` e `npm run build` (Aula 8). Como o caminho não mudou, não precisa refazer a Aula 10.
- **Se usou o git (Opção B):** dentro da pasta do projeto, rode:

  ```
  git pull
  npm install
  npm run build
  ```

Depois, reinicie o Claude Code.

Aula 14. Segurança: as regras de ouro

1. **A chave é secreta.** Nunca publique, nunca coloque em print, nunca envie em mensagem.
2. **Onde a chave fica guardada.** Com `--scope user`, o Claude Code grava a configuração no arquivo `.claude.json`, dentro da sua pasta de usuário. Esse arquivo contém a sua chave. **Não compartilhe esse arquivo** com ninguém.
3. **O arquivo `.env` é só para testes.** O projeto traz um modelo chamado `.env.example`. Ele serve para quem quer testar o servidor fora do Claude. O projeto já vem configurado para **nunca** enviar o `.env` verdadeiro ao GitHub.
4. **Vazou?** Entre em **https://aistudio.google.com/apikey**, **apague** a chave antiga, crie uma nova e refaça a Aula 10.
5. **Confira as respostas.** O Gemini traz fontes, mas continua sendo uma inteligência artificial. Trate a resposta como **um dado a verificar**: para informação importante (lei, prazo, valor, data), abra a fonte oficial e confirme.
6. **Código aberto.** Todo o código do projeto pode ser lido e auditado no repositório. Nada fica escondido.

Exercício de fixação

Responda de cabeça. O gabarito está logo abaixo.

1. Na analogia do escritório, o que é o servidor MCP?
2. Por que precisamos fechar e reabrir o Terminal depois de instalar o Node.js?
3. Qual comando cria a pasta `dist`?
4. Por que o `--transport stdio` fica entre o `--env` e o apelido `gemini-grounding`?
5. Sua chave vazou. Quais são os passos?

Gabarito

1. O ramal telefônico entre o Claude e o Gemini.
2. Porque o Terminal só reconhece programas instalados depois que é aberto de novo.
3. `npm run build`.
4. Porque, se o apelido vier logo depois do `--env`, o Claude o confunde com mais uma variável e recusa o comando.
5. Apagar a chave antiga no AI Studio, criar uma nova, remover a conexão (`claude mcp remove gemini-grounding --scope user`) e refazer a Aula 10.

Resumo de bolso

Para quem já fez uma vez e só quer relembrar:

```
# 1. Pré-requisito: Node.js 18+ (nodejs.org, versão LTS)
node --version

# 2. Chave gratuita: https://aistudio.google.com/apikey

# 3. Baixar e montar
git clone https://github.com/wall-e-assessor/gemini-grounding-mcp.git
```

```
cd gemini-grounding-mcp
npm install
npm run build

# 4. Conectar ao Claude Code (troque a chave e o caminho)
claude mcp add --scope user --env GEMINI_API_KEY=SUA_CHAVE_AQUI --transport stdio
    gemini-grounding -- node "/caminho/absoluto/gemini-grounding-mcp/dist/index.js"

# 5. Conferir
claude mcp list
```

Depois: reinicie o Claude Code e peça "faça uma busca na web com o Gemini sobre ...".

Créditos

Projeto, conexão e autoria: Hudson Travassos, **@pro.hudsontravassos**
Repositório oficial: https://github.com/wall-e-assessor/gemini-grounding-mcp
Licença: MIT (uso, cópia e modificação livres, mantido o aviso de autoria)

Se este manual te ajudou, mencione **@pro.hudsontravassos** ao compartilhar.

Fontes oficiais consultadas para este manual

- Documentação do Claude Code sobre MCP (sintaxe do `claude mcp add`, escopos, `list`, `get`, `remove`, `/mcp`): https://code.claude.com/docs/en/mcp
- Node.js, versões LTS: https://nodejs.org/en/about/previous-releases
- Google AI Studio, criação de chave: https://aistudio.google.com/apikey

Comandos e links conferidos em outubro de 2026. Ferramentas mudam: em caso de divergência, vale a documentação oficial.